FORMAL TECHNICAL DOCUMENT

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Enterprise Manufacturing, Job-Costing, Procurement, Commercial Sales, and Dual-Entry Financial Accounting System

SOFTWARE PRODUCT:
Traila ERP (Production Desktop Release)

DOCUMENT VERSION:
Version 1.0.0 (Master IEEE-830 Standard)

TARGET DEPLOYMENT:
Amjid Trailer Makers, Ring Road, Peshawar

PLATFORM ARCHITECTURE:
Windows 10 / 11 Desktop (Offline-First Kiosk)

CORE TECHNOLOGY STACK:
Flutter SDK (Dart ^3.10.8), Drift SQLite ORM v11

STATE MANAGEMENT:
Unidirectional Flutter BLoC (ERPCubit)

PRINTING SUBSYSTEM:
Dual-Engine: 58mm/80mm Thermal POS & A4 Spooler

DOCUMENT DISPATCH:
Integrated Shell & WhatsApp Automation

Prepared by: Senior Systems Engineering & Enterprise Architecture Team

Status: Approved Production Baseline

Date: October 2026

TABLE OF CONTENTS

1. INTRODUCTION Page 3

1.1 Purpose of the Document 3
1.2 Scope of the Software System 3
1.3 Intended Audience & Reading Suggestions 3
1.4 Document Conventions & Design Standards 4
1.5 References & Industry Guidelines 4

2. OVERALL DESCRIPTION Page 4

2.1 Product Perspective & Architectural Topology 4
2.2 System Modules Decomposition 5
2.3 User Classes & Operating Personas 5
2.4 Operating Environment & Platform Constraints 6
2.5 Design & Implementation Constraints 6
2.6 Assumptions & Dependencies 6

3. EXTERNAL INTERFACE REQUIREMENTS Page 7

3.1 User Interfaces & Visual Design System 7
3.2 Hardware Interfaces: Thermal POS & Laser Printers 7
3.3 Software Interfaces: Windows Desktop & File System 8
3.4 Communication Interfaces: WhatsApp Desktop Protocol 8

4. SYSTEM FEATURES & FUNCTIONAL REQUIREMENTS Page 9

4.1 Module 01: Application Lifecycle & Kiosk Window Control 9
4.2 Module 02: Drift Database Subsystem & Self-Healing Engine 9
4.3 Module 03: Executive Dashboard & Workshop Fleet Metrics 10
4.4 Module 04: Traila Production & Fleet Management 10
4.5 Module 05: Manufacturing Cost & Job-Costing Engine 11
4.6 Module 06: Supplier Procurement & Materials Registry 11
4.7 Module 07: Supplier Financial Ledger & Dual-Balance Settlements 12
4.8 Module 08: Labour Workforce & Piece-Rate Job Work 13
4.9 Module 09: Labour Financial Ledger & Wage Disbursements 13
4.10 Module 10: Finished Unit Sales & Commercial Contracts 14
4.11 Module 11: Sale Installments & Buyer Global Debt Consolidation 14
4.12 Module 12: Khata (Customer & Personal Credit Ledger) 15
4.13 Module 13: Cash Flow, Multi-Wallet Treasury & Transfers 16
4.14 Module 14: Profit & Loss (P&L) Financial Engine 16
4.15 Module 15: Document Generation & PDF Typography Engine 17
4.16 Module 16: Hardware Printing & Image Thresholding Subsystem 17
4.17 Module 17: External WhatsApp Automation & Customer Dispatch 18
4.18 Module 18: Workshop Branding & Application Profile Settings 18

5. NON-FUNCTIONAL REQUIREMENTS Page 19
5.1 Performance Requirements 19
5.2 Reliability & Fault Tolerance 19
5.3 Data Integrity & Constraints 19
5.4 Usability & Human Factors 20

6. DATA MODELS & RELATIONAL DATABASE SPECIFICATIONS Page 20
6.1 Entity-Relationship Overview 20
6.2 Complete Table Data Dictionary (18 Tables) 21

7. MATHEMATICAL FORMULATIONS & CORE ALGORITHMS Page 24
7.1 Traila Manufacturing Cost Aggregation 24
7.2 Supplier Dual-Balance Settlement Logic 24
7.3 Double-Entry Inter-Account Fund Transfer 25
7.4 Khata Bidirectional Synchronization Logic 25
7.5 Monochrome Thermal Image Luminance Thresholding 25

8. END-TO-END OPERATIONAL USE CASES Page 26
8.1 Use Case UC-01: Traila Manufacturing & Customer Sale 26
8.2 Use Case UC-02: Supplier Material Procurement with Partial Payout 27

9. REQUIREMENTS TRACEABILITY MATRIX & CODE MAP Page 28

10. GLOSSARY & DOMAIN TERMINOLOGY Page 29

1. INTRODUCTION

1.1 Purpose of the Document

This Software Requirements Specification (SRS) document provides a formal, comprehensive, and exhaustive specification of the software requirements for the **Traila ERP** application. It defines the functional behaviors, system architecture, database specifications, financial calculation models, hardware communication protocols, and non-functional engineering standards governing the system from initial boot to application teardown.

1.2 Scope of the Software System

Traila ERP is a purpose-built, offline-first Enterprise Resource Planning and Accounting System engineered specifically for heavy trailer/trolley manufacturers, agricultural vehicle fabricators, and body-maker commission workshops. The application automates the entire manufacturing, procurement, commercial sales, and financial management cycle, as deployed at *Amjid Trailer Makers, Main Hazar Khwani Ring Road, Peshawar*.

> **Key Operational Capabilities:** Traila ERP integrates serial-tracked vehicle fabrication tracking, precise piece-rate and raw material job-costing, supplier dual-balance (Payable vs Advance) ledgers, labour wage ledgers, commercial sales

> contracts with profit auditing, personal Khata (Udhaar Bahi) credit tracking, multi-wallet treasury cash-flow, 58mm/80mm thermal receipt and A4 PDF printing, and native Windows WhatsApp dispatch automation.

1.3 Intended Audience & Reading Suggestions

- **Software Engineers & Maintainers:** Consult Section 3 (System Architecture), Section 4 (Functional Requirements), Section 6 (Database Specifications), and Section 9 (Requirements Traceability Matrix) for implementation details.
- **Workshop Managers & Business Administrators:** Review Section 2 (Overall Description), Section 4 (Modules 3-14), and Section 8 (Operational Use Cases) to understand operational workflows.
- **Quality Assurance & Financial Auditors:** Reference Section 5 (Non-Functional Requirements) and Section 7 (Mathematical Formulations & Core Algorithms) for ledger reconciliation guarantees.

1.4 Document Conventions & Design Standards

Requirements are tagged with unique alphanumeric identifiers in the format `REQ-[Module]-[Number]` for functional requirements and `NFR-[Category]-[Number]` for non-functional requirements. Mathematical formulas are stated using standard arithmetic formulations and pseudo-code implementations.

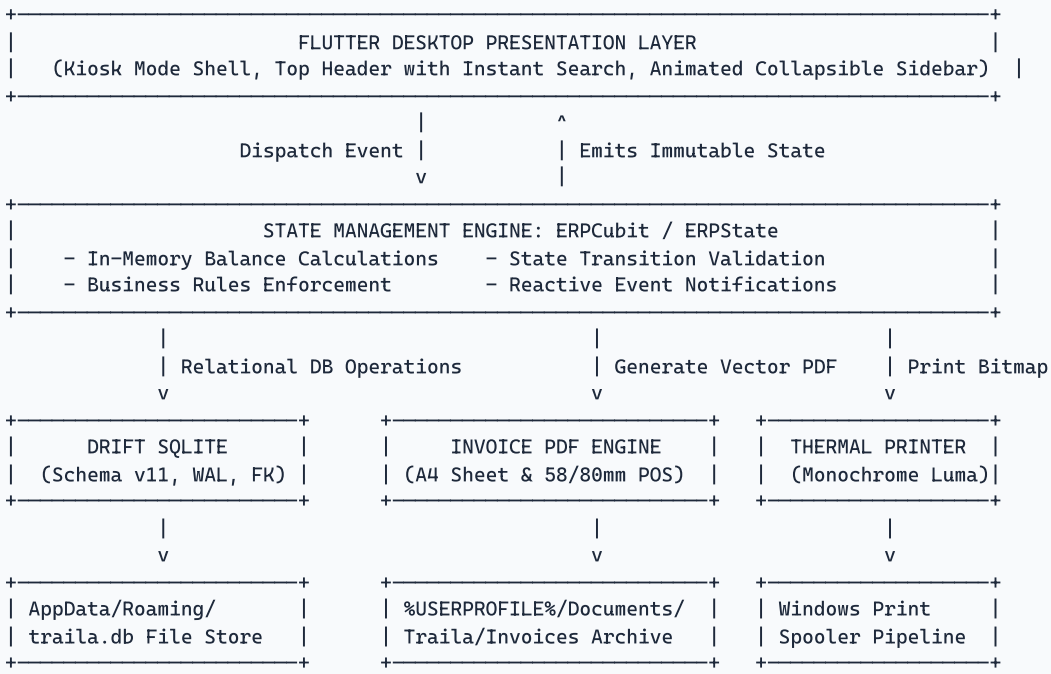
1.5 References & Industry Guidelines

- **IEEE Std 830-1998:** Recommended Practice for Software Requirements Specifications.
- **ISO/IEC/IEEE 29148:2018:** Systems and Software Engineering — Requirements Engineering.
- **SQLite 3 Specifications:** Relational engine specifications with PRAGMA foreign keys and ACID transactions.
- **Flutter & Drift ORM Documentation:** Reactive application architecture and type-safe database migrations.

2. OVERALL DESCRIPTION

2.1 Product Perspective & Architectural Topology

Traila ERP operates as a standalone, zero-cloud-dependent native 64-bit Windows desktop software application. It utilizes a unidirectional reactive state architecture orchestrated via **Flutter BLoC (`ERPCubit`)**, backed by an embedded SQLite database managed through **Drift ORM**.



2.2 System Modules Decomposition

Module ID	Module Name	Implementation File	Operational Scope
MOD-01	Application Shell & Kiosk Window	`main.dart`, `layout.dart`	Borderless kiosk window, collapsible sidebar, search header.
MOD-02	Drift Relational Storage & Healing	`database.dart`	SQLite v11 engine, PRAGMA foreign keys, self-healing.
MOD-03	Executive Dashboard & KPIs	`dashboard_screen.dart`	Fleet status summary, liquidity rollup, counter-party tallies.
MOD-04	Traila Unit Production Lifecycle	`traila_register_screen.dart`	Chassis registration, assembly state transitions (pending/ready/sold).
MOD-05	Manufacturing Cost Engine	`traila_history_screen.dart`	Job-costing roll-up: procurement materials + labour work + overheads.

Module ID	Module Name	Implementation File	Operational Scope
MOD-06	Supplier Procurement Registry	`supplier_management_screen.dart`	Supplier catalog, multi-item purchase bills, unit cost injection.
MOD-07	Supplier Ledger & Settlements	`supplier_profile_screen.dart`	Dual-balance debt/advance engine, split payments, print statements.
MOD-08	Labour Workforce & Work Orders	`labour_management_screen.dart`	Worker directory, task logging, automatic unit cost linkage.
MOD-09	Labour Ledger & Wage Settlements	`labour_profile_screen.dart`	Wage vouchers, wallet deductions, balance reconciliation.
MOD-10	Commercial Sales & Unit Handoff	`sales_management_screen.dart`	Sale execution, profit computation, CNIC capture, downpayment.
MOD-11	Sale Installments & Debt Aging	`sale_detail_screen.dart`	Periodic installment collection, consolidated buyer debt tracking.
MOD-12	Khata Credit Ledger (Udhaar Bahi)	`khata_management_screen.dart`	Informal counter-party ledger, optional cash wallet sync.
MOD-13	Multi-Wallet Treasury & Transfers	`cash_flow_screen.dart`	Wallet management, atomic double-entry money transfers.
MOD-14	P&L Financial Engine & Reporting	`expense_revenue_screen.dart`	Temporal filtering (Monthly/Yearly/Custom), revenue vs expense.
MOD-15	Document Generation & Typography	`invoice_pdf_service.dart`	18 distinct vector PDF document layouts (A4 and 58/80mm roll).
MOD-16	Hardware Printing Subsystem	`thermal_printer_service.dart`	Monochrome luminance thresholding for 203 DPI thermal heads.
MOD-17	WhatsApp & Shell Automation	`whatsapp_service.dart`	PDF disk caching, URI launch (`whatsapp://`), Explorer highlight.
MOD-18	Workshop Branding & Profile	`app_profile_screen.dart`	Business branding metadata, owner contacts, printer page defaults.

2.3 User Classes & Operating Personas

- **Workshop General Manager:** Oversees trailer manufacturing progress, sets sale contracts, views executive profit margins, and manages workshop profile settings.
- **Workshop Accountant / Cashier:** Records material purchase bills, logs labour piece-rate entries, collects customer installment payments, manages cash/bank wallets, and prints receipts.
- **Storekeeper / Procurement Supervisor:** Manages raw material inventory bills (steel sheets, axles, leaf springs) and links material costs to trailer IDs.

2.4 Operating Environment & Platform Constraints

- **Operating System:** Microsoft Windows 10 (Build 19041+) and Windows 11 64-bit.

- **Hardware Footprint:** Minimal hardware footprint requiring x86_64 CPU (2.0 GHz+), 4 GB RAM, 500 MB free disk space.
- **Database Storage:** Stored locally at `%APPDATA%Roaming<app-id> raila.db`.
- **Invoice Archive:** Automated file export to `%USERPROFILE%DocumentsTrailaInvoices`.

3. EXTERNAL INTERFACE REQUIREMENTS

3.1 User Interfaces & Visual Design System

The application shell executes in **Kiosk Mode** (borderless, title bar controls removed, centered and auto-maximized to full display bounds).

- **Layout Structure:** Responsive horizontal split featuring a collapsible dark slate navigation sidebar (`260px` expanded, `80px` collapsed, background `#1E293B`) and a light content canvas (background `#F8FAFC`).
- **Ergonomic Spacing:** Standard `29px` (approx. 0.3-inch) separation between navigation and active module views.
- **Header Bar:** Fixed `70px` header housing a `300px` search text field that instantly updates `ERPState.searchQuery` , dynamically filtering tables and lists without reload buttons.
- **Color Palette:** Primary seed `#2563EB` (Royal Blue), background `#FFFFFF` , cards border `#E2E8F0` with `12px` rounded corners.
- **Typography:** Google Fonts `Inter` applied globally across all UI widgets.

3.2 Hardware Interfaces: Thermal POS & Office Printers

- **Thermal Point-of-Sale (POS) Printers:** Native spooling to 58mm (width `164 pt`) and 80mm (width `226 pt`) ESC/POS-compatible continuous receipt printers.
- **Office Laser / Inkjet Printers:** Native spooling to standard A4 paper format (210mm x 297mm) via the Windows Print Spooler.
- **Luminance Thresholding:** High-resolution color logos are dynamically downscaled to `300px` width, converted to 8-bit grayscale, and thresholded ($L > 0.6$ implies 255$, else 0) to eliminate thermal print head ink-bleed.

3.3 Software Interfaces: Windows Desktop & Shell

- **Window Management:** Managed via `window_manager` for borderless kiosk execution.
- **File System Access:** `path_provider` maps persistent database storage to application support paths.
- **Shell Spooling:** Direct process execution via `Process.run('explorer.exe', ['/select,', filePath])` to bring the file manager to the foreground with the active invoice highlighted.

3.4 Communication Interfaces: WhatsApp Desktop Protocol

The application automates document delivery via WhatsApp without external paid APIs:

1. The vector PDF is generated and serialized to `%USERPROFILE%DocumentsTrailaInvoices<filename>.pdf` .
2. The telephone number is sanitized and converted to international E.164 format (Pakistani numbers starting with `03 ...` or 10 digits are prepended with country code `92`).
3. The application executes `cmd.exe /c start whatsapp:// send?phone=<sanitizedNumber>` , opening WhatsApp Desktop to the client chat.
4. Simultaneously, Windows Explorer opens with the generated PDF selected for immediate drag-and-drop into the chat window.

4. SYSTEM FEATURES & FUNCTIONAL REQUIREMENTS

4.1 Module 01: Application Lifecycle & Kiosk Window Control

- **REQ-01-01:** The application shall initialize the `window_manager` plugin on launch, strip all standard OS window decorations, center the window, and maximize it into kiosk mode.
- **REQ-01-02:** The sidebar menu shall provide an `Exit App` action that invokes `windowManager.close()` to terminate the application cleanly.
- **REQ-01-03:** The main shell shall manage page transitions via an unscrollable `PageView` controlled by `currentModuleIndex` with smooth curve animations (`300ms` duration).

4.2 Module 02: Drift Relational Storage & Self-Healing Engine

- **REQ-02-01:** The database shall execute under schema version `11` with foreign key enforcement active (`PRAGMA foreign_keys = ON`).
- **REQ-02-02 (Self-Healing Balances):** During `loadData()` , if any supplier or labourer record has `remainingBalance == 0` , `openingBalance > 0` , and zero prior transactions, the system shall automatically repair the database by setting `remainingBalance = openingBalance` .
- **REQ-02-03 (Legacy Invoice Backfill):** The system shall scan legacy records and generate backfilled `OPEN-BAL` invoices for any entity possessing an opening balance lacking an invoice voucher.
- **REQ-02-04 (Legacy Sales Downpayment Backfill):** The system shall inspect historical sales where `paidAmount > 0` lacking downpayment vouchers, automatically generating synthetic records (`DP-<saleId>-BACKFILL`) in the payments ledger.

4.3 Module 03: Executive Dashboard & Workshop Fleet Metrics

- **REQ-03-01:** The dashboard shall calculate and render live counts of manufactured trailers categorized into three mutually exclusive states: `Pending Trailers` , `Ready Trailers` , and `Sold Trailers` .
- **REQ-03-02:** The dashboard shall calculate aggregate available cash across all registered accounts: $ ext{Total Balance} = sum ext{AccountBalance}_i$.
- **REQ-03-03:** Active counter-party summaries shall display the total count of registered suppliers and total workshop labour workforce.
- **REQ-03-04:** Quick action buttons shall enable single-click navigation to Traila Register (Index 1), Procurement (Index 2), Labour Work (Index 3), and Sales (Index 4).

4.4 Module 04: Traila Production & Fleet Management

- **REQ-04-01:** The system shall support registering new trailer units with a mandatory manual identifier (e.g., `T1` , `T-102`) and Model Name.
- **REQ-04-02:** Newly created trailers shall automatically be assigned status `TrailaStatus.pending` with initial `totalManufacturingCost = 0.0` .
- **REQ-04-03:** The system shall support modifying trailer metadata (ID, Model Name, Registration Date).

- **REQ-04-04:** The system shall allow transitioning trailer status between `pending` (under construction), `ready` (available for commercial sale), and `sold`.
- **REQ-04-05:** Deletion of a trailer unit shall cascade delete all associated expenses, preventing orphan records.

4.5 Module 05: Manufacturing Cost & Job-Costing Engine

- **REQ-05-01:** The Unit Manufacturing Cost ($C_{ ext{Traila}}$) shall be calculated dynamically as: $$C_{ ext{Traila}} = sum M_{ ext{Materials}} + sum W_{ ext{Labour}} + sum E_{ ext{Overhead}}$$
- **REQ-05-02:** Any procurement bill or labour entry linked to a trailer ID shall trigger `_syncTrailaTotalCost(trailaId)`, updating the persisted cost in the `Trailas` table.
- **REQ-05-03:** Manual overhead expenses (e.g., permits, specialized transport) can be added directly to any trailer unit via the Traila History interface.

4.6 Module 06: Supplier Procurement & Materials Registry

- **REQ-06-01:** The system shall maintain a supplier directory capturing Manual ID, Name, Phone Number, Shop Name, Market Location, Address, WhatsApp Number, and Initial Opening Balance.
- **REQ-06-02:** Purchase bills shall capture: Bill Number (auto-generated `INV-SP-XXXX` if blank), Bill Date, Target Traila ID (optional), Multi-item lines (Name, Quantity, Unit Price, Total), Upfront Payment Amount, and Source Account.
- **REQ-06-03:** Line total and bill total shall compute as: $ ext{LineTotal} = ext{Qty} imes ext{UnitPrice}$, $ ext{BillTotal} = sum ext{LineTotal}_i$.

4.7 Module 07: Supplier Financial Ledger & Dual-Balance Settlements

- **REQ-07-01 (Dual-Balance Logic):** Supplier net balance evaluates according to: $$ ext{Net} = ext{OpeningBalance} + sum ext{Purchases} - sum ext{Payments}$$ If $ ext{Net} ge 0 implies ext{RemainingBalance} = ext{Net}$ (Payable debt owed to supplier).
  If $ ext{Net} < 0 implies ext{AdvanceBalance} = | ext{Net}|$ (Advance credit balance held by supplier).
- **REQ-07-02 (Snapshot Integrity):** Invoice generation shall snapshot the pre-transaction balance from the database and store it directly in `invoices.previousBalance`.
- **REQ-07-03 (Split Payments):** The system shall support split-disbursements across multiple accounts simultaneously (e.g., Rs. 50,000 Cash + Rs. 150,000 Bank).
- **REQ-07-04:** Every supplier payment disbursement shall post a debit transaction (`type: take`) to the designated financial wallet.

4.8 Module 08: Labour Workforce & Piece-Rate Job Work

- **REQ-08-01:** The system shall maintain a labour workforce directory recording Worker ID, Name, Phone, WhatsApp, Home Address, and Opening Balance.
- **REQ-08-02:** Work orders shall capture: Entry Number (auto-generated `INV-LW-XXXX` if blank), Target Traila ID, Task Description (e.g., "Chassis welding", "Double axle fitting"), Piece-rate Charge, and optional Upfront Wage Payment.
- **REQ-08-03:** Upfront payments logged during work order entry shall automatically generate a linked payment voucher (`LPAY-…`), debit the chosen wallet, and adjust the worker's balance.

4.9 Module 09: Labour Financial Ledger & Wage Disbursements

- **REQ-09-01:** Worker payable wage balance evaluates according to: $$ ext{Wage Due} = ext{OpeningBalance} + sum ext{Work Charges} - sum ext{Wages Paid}$$
- **REQ-09-02:** Wage disbursement vouchers shall generate sequential receipt numbers (`INV-L-PAY-XXXXX`).
- **REQ-09-03:** Deleting or modifying a wage payment shall execute `_syncLabourBalances(labourId)` to recalculate running balances.

4.10 Module 10: Finished Unit Sales & Commercial Contracts

- **REQ-10-01:** Only trailer units possessing status `TrailaStatus.ready` shall be eligible for commercial sale.
- **REQ-10-02:** Sale contract creation shall capture: Ready Unit Selection, Buyer Profiling (Name, Phone, CNIC, Address, WhatsApp), Agreed Sale Price ($P_{ ext{Sale}}$), Initial Downpayment, Target Wallet, and Sale Date.
- **REQ-10-03 (Internal Profit Engine):** The system shall compute gross operational unit profit: $$ ext{Profit} = P_{ ext{Sale}} - C_{ ext{Manufacturing}}$$ This figure is strictly retained for management auditing and shielded from customer documents.
- **REQ-10-04:** Contract execution shall update the trailer's status in `Trailas` to `'sold'` and store `salePrice` .
- **REQ-10-05:** Sales shall generate unique commercial sequence receipts formatted as `SALE-XXXXX` .

4.11 Module 11: Sale Installments & Buyer Global Debt Consolidation

- **REQ-11-01 (Consolidated Debt Tracking):** When generating a sale invoice or installment receipt, the system shall aggregate all unpaid debts from all other trailers purchased by the same buyer: $$ ext{Global Previous Debt} = sum_{s in ext{Sales}_{ ext{Buyer}} setminus {s_{ ext{current}}}} ext{RemainingBalance}(s)$$ $$ ext{Gross Total Due} = P_{ ext{Sale}} + ext{Global Previous Debt}$$
- **REQ-11-02:** Subsequent installment payments shall be tracked in the payments ledger, updating remaining debt and posting an incoming cash transaction (`type: add`).
- **REQ-11-03:** Sales Management shall provide segregated views: `Ready for Sale` , `Partial / Pending` , and `Fully Paid` .

4.12 Module 12: Khata (Customer & Personal Credit Ledger)

- **REQ-12-01:** The system shall provide an independent Khata subsystem for tracking general counter-party credits, advances, and customer accounts (Udhaar Bahi).
- **REQ-12-02:** Transactions shall be categorized as `KhataTransactionType.cashOut` (debit, increases debt owed to business) or `KhataTransactionType.cashIn` (credit, decreases debt).
- **REQ-12-03 (Account Sync Toggle):** Each transaction features a `syncToAccount` flag: - If enabled: Debits or credits the designated commercial account wallet concurrently.
  - If disabled: Updates the personal ledger without altering physical cash balances.
- **REQ-12-04:** Khata running balance calculates as: $ ext{Balance} = ext{Opening} + sum ext{CashOut} - sum ext{CashIn}$.

4.13 Module 13: Cash Flow, Multi-Wallet Treasury & Transfers

- **REQ-13-01:** The system shall support creating multiple commercial liquidity wallets (e.g., `Cash In Hand` , `Meezan Bank` , `HBL` , `EasyPaisa`).

- **REQ-13-02:** Inter-account fund transfers shall execute as an atomic transaction creating two linked entries: 1. A debit entry on source account (`type: transfer` , `id: TRF-...-OUT`).
  2. A credit entry on destination account (`type: transfer` , `id: TRF-...-IN`).
  3. Both entries share an identical `linkedTransferId` .
- **REQ-13-03:** Modifying or deleting a transfer transaction shall automatically reconcile the reciprocal twin entry.

4.14 Module 14: Profit & Loss (P&L) Financial Engine

- **REQ-14-01:** The P&L engine shall filter financial metrics across temporal boundaries: `All Time` , `Monthly` , `Yearly` , and `Custom Date Range` .
- **REQ-14-02:** Financial engine metrics evaluation: $$ ext{Total Revenue} = sum_{s in ext{Sales} cap ext{Range}} s. ext{salePrice}$$ $$ ext{Total Expenses} = sum_{t in ext{Trailas} cap ext{Range}} t. ext{totalManufacturingCost}$$ $$ ext{Net Operating Profit} = ext{Total Revenue} - ext{Total Expenses}$$

4.15 Module 15: Document Generation & PDF Typography Engine

- **REQ-15-01:** The system shall provide vector PDF document generation (via `InvoicePdfService`) for 18 distinct document types:
  1. `generateSupplierPurchasePdf` (A4 Purchase Bill)
  2. `generateSupplierPaymentPdf` (A4 Supplier Payment Voucher)
  3. `generateLabourWorkPdf` (A4 Labour Work Order)
  4. `generateLabourPaymentPdf` (A4 Labour Wage Voucher)
  5. `generateSalePdf` (A4 Commercial Vehicle Sale Invoice)
  6. `generateStatementPdf` (A4 Running Ledger Statement)
  7. `generateKhataPdf` (A4 Khata Customer Statement)
  8. `generateFinancialReportPdf` (A4 P&L and Balance Summary)
  9. `generateCashFlowReportPdf` (A4 Treasury & Wallet Audit)
  10. `generateThermalSalePdf` (58/80mm Sale Receipt)
  11. `generateThermalPurchasePdf` (58/80mm Purchase Slip)
  12. `generateThermalPaymentPdf` (58/80mm Payment Slip)
  13. `generateThermalSalePaymentPdf` (58/80mm Installment Receipt)
  14. `generateThermalWorkPdf` (58/80mm Work Slip)
  15. `generateThermalKhataPdf` (58/80mm Khata Voucher)
  16. `generateThermalSummaryPdf` (58/80mm Executive Summary)
  17. `generateThermalStatementPdf` (58/80mm Account Ledger Slip)
  18. `generateThermalTransactionPdf` (58/80mm Transaction Slip)

4.16 Module 16: Hardware Printing & Image Thresholding Subsystem

- **REQ-16-01:** The system shall interface directly with the Windows Print Spooler via `printing: ^5.14.2` .
- **REQ-16-02:** Color bitmap assets printed to thermal paper shall be processed via monochrome luminance thresholding ($L le 0.6 implies ext{Black}$, else $ ext{White}$) to guarantee sharp printing without smudges.
- **REQ-16-03:** The system shall provide an `autoPrint` configuration toggle to trigger printing automatically upon completing a sale.

4.17 Module 17: External WhatsApp Automation & Customer Dispatch

- **REQ-17-01:** Document sharing shall save the generated vector PDF to `%USERPROFILE%DocumentsTrailaInvoices<filename>.pdf`.
- **REQ-17-02:** The system shall launch WhatsApp Desktop via URI: `whatsapp://send?phone=<sanitizedNumber>`.
- **REQ-17-03:** The system shall execute `explorer.exe /select,"<filePath>"` to open Windows Explorer with the generated invoice highlighted for drag-and-drop delivery.

4.18 Module 18: Workshop Branding & Application Profile Settings

- **REQ-18-01:** The application shall store corporate letterhead metadata in the `AppProfiles` table: Business Name, Address, Owner Name, Telephone, and WhatsApp Number.
- **REQ-18-02:** All branding parameters shall dynamically populate PDF document headers and thermal receipts, falling back to default workshop constants if unconfigured.

5. NON-FUNCTIONAL REQUIREMENTS

5.1 Performance Requirements

- **NFR-PERF-01 (Query Execution Latency):** Local SQLite database queries executed by `loadData()` shall execute in under `150ms` for datasets up to 50,000 records.
- **NFR-PERF-02 (PDF Rendering Speed):** Vector PDF rendering for multi-page statements or thermal receipts shall complete in under `350ms`.
- **NFR-PERF-03 (UI Frame Rate):** The user interface shall maintain a consistent `60 FPS` on dual-core x86_64 desktop systems with 4 GB RAM. Global search query filtering shall evaluate without keystroke lag.

5.2 Reliability & Fault Tolerance

- **NFR-REL-01 (ACID Transaction Atomicity):** Multi-table operations (e.g., trailer sales touching 7 tables) shall execute inside an atomic `_db.transaction()`. Any unexpected runtime failure shall cause a complete rollback with zero corrupted data.
- **NFR-REL-02 (Offline-First Execution):** The system shall execute 100% of all computational, database, reporting, and printing workflows offline with zero cloud dependency.

5.3 Data Integrity & Security

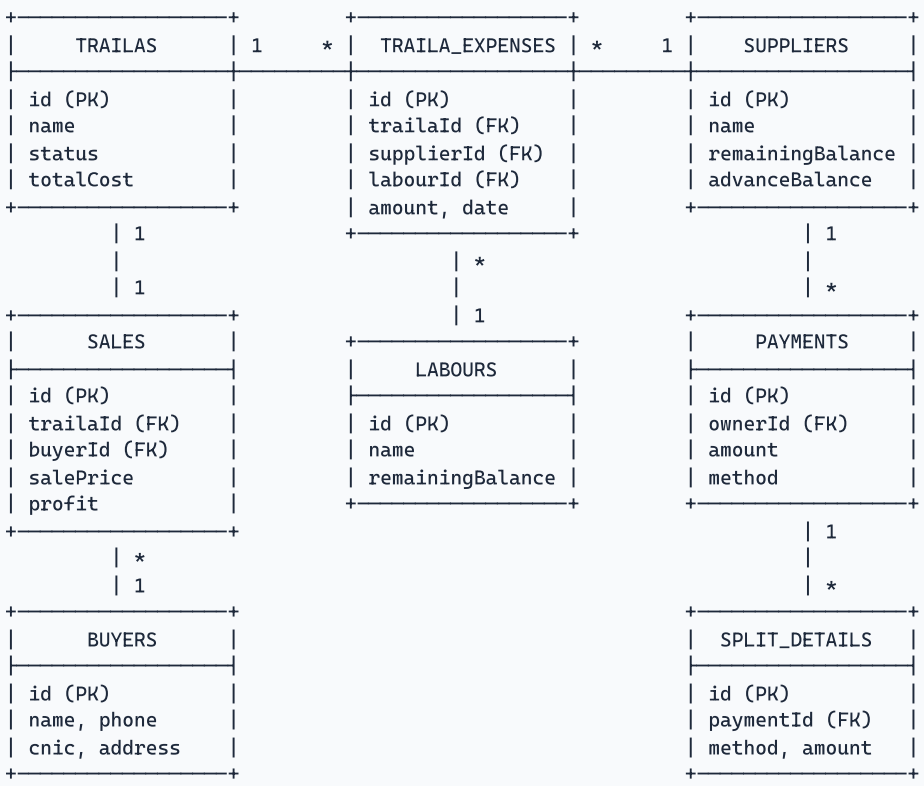
- **NFR-SEC-01 (Foreign Key Enforcement):** Foreign key constraints (`PRAGMA foreign_keys = ON`) shall be enforced on every database connection open.
- **NFR-SEC-02 (Cascading Integrity):** Deleting a trailer shall cascade delete child `TrailaExpenses` and set linked invoices to null. Deleting a sale shall cascade delete linked installment records.
- **NFR-SEC-03 (SQL Injection Immunity):** All database queries shall utilize Drift's type-safe parameterized query builder, guaranteeing complete immunity to SQL injection vulnerabilities.

5.4 Usability & Human Factors

- **NFR-USE-01:** The application shall operate in kiosk mode to maximize screen area for workshop environments. All numeric inputs shall feature automatic validation and formatting.
- **NFR-USE-02:** Financial amounts shall consistently display the currency prefix `Rs.` with clean number formatting.

6. DATA MODELS & RELATIONAL DATABASE SPECIFICATIONS

6.1 Entity-Relationship Architecture



6.2 Relational Schema Data Dictionary (18 Tables)

Table 1: Suppliers

Column Name	Drift Type	Constraints	Default	Description
`id`	TextColumn	PRIMARY KEY	-	Manual supplier identifier (e.g. `S-01`).
`name`	TextColumn	NOT NULL	-	Supplier legal/contact name.
`phone`	TextColumn	NOT NULL	`''`	Primary phone number.
`shopName`	TextColumn	NOT NULL	`''`	Business trading name.
`location`	TextColumn	NOT NULL	`''`	Market location / district.
`whatsappNumber`	TextColumn	NOT NULL	`''`	WhatsApp international phone number.
`address`	TextColumn	NOT NULL	`''`	Physical warehouse address.

Column Name	Drift Type	Constraints	Default	Description
`registrationDate`	DateTimeColumn	NOT NULL	-	Supplier onboarding date.
`openingBalance`	RealColumn	NOT NULL	`0.0`	Seed balance at system initialization.
`totalAmount`	RealColumn	NOT NULL	`0.0`	Cumulative sum of purchase invoices.
`totalPaid`	RealColumn	NOT NULL	`0.0`	Cumulative payments issued to supplier.
`remainingBalance`	RealColumn	NOT NULL	`0.0`	Net payable balance due to supplier.
`advanceBalance`	RealColumn	NOT NULL	`0.0`	Advance credit balance held by supplier.
`createdAt`	DateTimeColumn	NOT NULL	`currentDateAndTime`	Record insertion timestamp.

Table 2: Labours

Column Name	Drift Type	Constraints	Default	Description
`id`	TextColumn	PRIMARY KEY	-	Manual worker identifier (e.g. `L-01`).
`name`	TextColumn	NOT NULL	-	Worker name.
`phone`	TextColumn	NOT NULL	`''`	Mobile telephone number.
`whatsappNumber`	TextColumn	NOT NULL	`''`	WhatsApp contact telephone.
`address`	TextColumn	NOT NULL	`''`	Home residential address.
`registrationDate`	DateTimeColumn	NOT NULL	-	Date registered.
`openingBalance`	RealColumn	NOT NULL	`0.0`	Seed payable balance.
`totalPayable`	RealColumn	NOT NULL	`0.0`	Cumulative work charges logged.
`totalPaid`	RealColumn	NOT NULL	`0.0`	Cumulative wages paid.
`remainingBalance`	RealColumn	NOT NULL	`0.0`	Net wages balance currently due.
`createdAt`	DateTimeColumn	NOT NULL	`currentDateAndTime`	Record insertion timestamp.

Table 3: Trailas

Column Name	Drift Type	Constraints	Default	Description
`id`	TextColumn	PRIMARY KEY	-	Chassis / unit serial ID (e.g. `T1`).
`name`	TextColumn	NOT NULL	-	Trailer model / specifications.
`registrationDate`	DateTimeColumn	NOT NULL	-	Commencement date.
`status`	TextColumn	NOT NULL	-	Status enum: `pending`, `ready`, `sold`.
`totalManufacturingCost`	RealColumn	NOT NULL	`0.0`	Total aggregated job cost.
`salePrice`	RealColumn	NULLABLE	-	Final sold price.

Column Name	Drift Type	Constraints	Default	Description
`createdAt`	DateTimeColumn	NOT NULL	`currentDateAndTime`	Record insertion timestamp.

Table 4: TrailaExpenses

Column Name	Drift Type	Constraints	Default	Description
`id`	TextColumn	PRIMARY KEY	-	Expense record ID.
`trailaId`	TextColumn	NULLABLE, FK `Trailas(id)` ON DELETE CASCADE	-	Associated trailer unit.
`type`	TextColumn	NOT NULL	-	Type: `Material`, `Labour`, `General`.
`description`	TextColumn	NOT NULL	-	Description of expenditure.
`amount`	RealColumn	NOT NULL	-	Monetary cost.
`date`	DateTimeColumn	NOT NULL	-	Transaction date.
`supplierId`	TextColumn	NULLABLE, FK `Suppliers(id)`	-	Linked supplier ID.
`labourId`	TextColumn	NULLABLE, FK `Labours(id)`	-	Linked labourer ID.
`createdAt`	DateTimeColumn	NOT NULL	`currentDateAndTime`	Record insertion timestamp.

Table 5: Buyers

Column Name	Drift Type	Constraints	Default	Description
`id`	TextColumn	PRIMARY KEY	-	Buyer unique ID.
`name`	TextColumn	NOT NULL	-	Full buyer name.
`phone`	TextColumn	NULLABLE	-	Contact telephone.
`cnic`	TextColumn	NULLABLE	-	National Identity Card number.
`address`	TextColumn	NULLABLE	-	Residential / business address.
`whatsappNumber`	TextColumn	NULLABLE	-	WhatsApp phone number.
`createdAt`	DateTimeColumn	NOT NULL	`currentDateAndTime`	Record insertion timestamp.

Table 6: Sales

Column Name	Drift Type	Constraints	Default	Description
`id`	TextColumn	PRIMARY KEY	-	Sale identifier (`S-...`).
`trailaId`	TextColumn	NOT NULL, FK `Trailas(id)` ON DELETE CASCADE	-	Associated trailer unit.

Column Name	Drift Type	Constraints	Default	Description
`buyerId`	TextColumn	NOT NULL, FK `Buyers(id)`	-	Purchaser entity ID.
`salePrice`	RealColumn	NOT NULL	-	Gross agreed sale price.
`manufacturingCost`	RealColumn	NOT NULL	`0.0`	Production cost at sale time.
`profit`	RealColumn	NOT NULL	`0.0`	Calculated gross profit margin.
`saleDate`	DateTimeColumn	NOT NULL	-	Contract finalization date.
`paidAmount`	RealColumn	NOT NULL	-	Total paid to date.
`remainingBalance`	RealColumn	NOT NULL	-	Outstanding balance due on this unit.
`receiptNumber`	TextColumn	NOT NULL	-	Receipt sequence code (`SALE-XXXXX`).
`globalPreviousBalance`	RealColumn	NOT NULL	`0.0`	Buyer debt across all other units.
`createdAt`	DateTimeColumn	NOT NULL	`currentDateAndTime`	Record insertion timestamp.

Table 7: Invoices & InvoiceItems

`Invoices` : Stores formal vouchers for all 6 transaction types (`purchase` , `payment` , `labourWork` , `labourPayment` , `sale` , `salePayment`). Contains `previousBalance` and `remainingBalance` snapshots to ensure permanent historical accuracy.
`InvoiceItems` : Line item details (`productName` , `quantity` , `unitPrice` , `lineTotal`) linked via `invoiceId` with cascading deletion.

Table 8: Payments & SplitDetails

`Payments` : Tracks disbursements and collections. Contains `receiptNumber` , `amount` , `date` , `method` , `previousBalance` , and `updatedBalance` .
`SplitDetails` : Child table supporting multi-account payments (e.g. Rs. 50,000 Cash + Rs. 150,000 Bank), linked via `paymentId` with cascading deletion.

Table 9: Accounts & Transactions

`Accounts` : Treasury wallets (`id` , `name` , `balance` , `type` : cash/bank/wallet).
`Transactions` : Double-entry audit ledger storing every balance alteration (`type` : `add` , `take` , `transfer` , `opening`), linked transfer references, person name, description, and timestamps.

Table 10: Khatas & KhataItems

`Khatas` : Informal counter-party credit folders (`personName` , `openingBalance` , `totalAmount` , `paidAmount` , `remainingBalance`).

`KhataItems`: Individual debit/credit lines (`itemName`, `price`, `type`: cashIn/cashOut, `paymentMethod`, `date`).

Table 11: AppProfiles & PrinterSettings

`AppProfiles`: Workshop trading metadata (`businessName`, `ownerName`, `phoneNumber`, `address`, `whatsappNumber`).
`PrinterSettings`: Hardware configuration (`paperSize`: 58mm/80mm, `autoCut`, `autoPrint`, `connectionType`).

7. MATHEMATICAL FORMULATIONS & CORE ALGORITHMS

7.1 Traila Manufacturing Cost Aggregation Algorithm

$$C_{ \text{Traila}} = \sum_{m \in M} \text{Amount}_m + \sum_{w \in W} \text{Amount}_w + \sum_{e \in E} \text{Amount}_e$$

```
Future<void> _syncTrailaTotalCost(String trailaId) async {
  final expenses = await (_db.select(_db.trailaExpenses)
    ..where((t) => t.trailaId.equals(trailaId))).get();
  final totalCost = expenses.fold<double>(0.0, (sum, e) => sum + e.amount);

  await (_db.update(_db.trailas)..where((t) => t.id.equals(trailaId))).write(
    TrailasCompanion(totalManufacturingCost: Value(totalCost)),
  );
}
```

7.2 Supplier Dual-Balance Settlement Logic

$$\text{NetBalance} = \text{OpeningBalance} + \sum \text{Purchases} - \sum \text{Payments} \\ \text{RemainingBalance} = \max(0.0, \text{NetBalance}) \quad \text{(Payable Debt)} \\ \text{AdvanceBalance} = \max(0.0, -\text{NetBalance}) \quad \text{(Advance Credit)}$$

```
final net = supplier.openingBalance + totalAmount - totalPaid;
final remaining = net > 0 ? net : 0.0;
final advance = net < 0 ? -net : 0.0;

await (_db.update(_db.suppliers)..where((t) => t.id.equals(supplierId))).write(
  SuppliersCompanion(
    totalAmount: Value(totalAmount),
    totalPaid: Value(totalPaid),
    remainingBalance: Value(remaining),
    advanceBalance: Value(advance),
  ),
);
```

7.3 Double-Entry Inter-Account Fund Transfer

$$B_{ \text{Source, New}} = B_{ \text{Source, Current}} - \text{Amount} \\ B_{ \text{Dest, New}} = B_{ \text{Dest, Current}} + \text{Amount}$$

```
await _db.transaction(() async {
  // 1. Post Debit to Source Account
  await _db.into(_db.transactions).insert(TransactionsCompanion.insert(
    id: '$transferId-OUT', accountName: fromAccount.name,
    type: TransactionType.transfer.name, amount: amount,
    person: toAccount.name, description: 'Transfer OUT to ${toAccount.name}',
    date: date, category: 'Transfer', linkedTransferId: Value(transferId),
  ));
  // 2. Post Credit to Destination Account
  await _db.into(_db.transactions).insert(TransactionsCompanion.insert(
    id: '$transferId-IN', accountName: toAccount.name,
    type: TransactionType.transfer.name, amount: amount,
    person: fromAccount.name, description: 'Transfer IN from ${fromAccount.name}',
    date: date, category: 'Transfer', linkedTransferId: Value(transferId),
  ));
  // 3. Atomically update balances
  await (_db.update(_db.accounts)..where((t) => t.name.equals(fromAccount.name)))
      .write(AccountsCompanion(balance: Value(fromAccount.balance - amount)));
  await (_db.update(_db.accounts)..where((t) => t.name.equals(toAccount.name)))
      .write(AccountsCompanion(balance: Value(toAccount.balance + amount)));
});
```

7.4 Monochrome Thermal Image Luminance Thresholding

> $$L = \frac{0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B}{255.0} \\ P_{\text{Output}} = \begin{cases} 255 \text{ (White)}, & L > 0.6 \\ 0 \text{ (Pure Black)}, & L \le 0.6 \end{cases}$$

```
for (final pixel in processedImage) {
  final double l = (0.299 * pixel.r + 0.587 * pixel.g + 0.114 * pixel.b) / 255.0;
  if (l > 0.6) {
    pixel.r = 255; pixel.g = 255; pixel.b = 255;
  } else {
    pixel.r = 0; pixel.g = 0; pixel.b = 0;
  }
}
```

8. END-TO-END OPERATIONAL USE CASES

8.1 Use Case UC-01: Traila Manufacturing & Customer Sale

Use Case ID:	UC-01
Title:	Complete Traila Manufacturing from Assembly to Commercial Sale
Actors:	Workshop Manager, Cashier/Accountant, Buyer
Pre-conditions:	Application initialized, accounts provisioned, suppliers and workers registered.
Main Success Scenario:	1. Manager registers new Traila ID `T-101` ("14-Wheeler Heavy Haulage"). System sets status to `pending`. 2. Procurement buys steel plates from Supplier `S-01` (Rs. 350,000), linking to Traila `T-101`. Unit cost updates to Rs. 350,000. 3. Labour worker `L-03` logs chassis welding work (Rs. 100,000), linked to `T-101`. Unit cost updates to Rs. 450,000. 4. Assembly finishes: Manager toggles trailer status to `ready`. 5. Customer purchases `T-101` for Rs. 650,000, paying Rs. 200,000 Cash downpayment. 6. System validates: Profit = Rs. 200,000; Remaining Debt = Rs. 450,000; Trailer status becomes `sold`; Cash account receives Rs. 200,000. 7. System triggers thermal POS receipt printing and launches WhatsApp Desktop with PDF file highlighted in Explorer.
Post-conditions:	Unit status marked `sold`, profit recorded, treasury updated, customer receipt delivered.

8.2 Use Case UC-02: Supplier Procurement with Partial Upfront Settlement

Use Case ID:	UC-02
Title:	Raw Material Purchase with Partial Cash Settlement
Actors:	Storekeeper / Cashier, Supplier
Main Success Scenario:	1. Cashier opens Supplier Profile `S-02 (Khyber Axles)`. Current balance: Rs. 0. 2. Cashier enters purchase bill for 2 Heavy Axles at Rs. 100,000 each (Total: Rs. 200,000). 3. Cashier logs an upfront payment of Rs. 80,000 from `Meezan Bank` account. 4. System atomically executes: creates invoice `INV-SP-0012`; snapshots previous balance (Rs. 0); creates payment record (Rs. 80,000); debits Meezan Bank account by Rs. 80,000; updates supplier debt to Rs. 120,000. 5. System prints 80mm thermal purchase voucher and saves A4 PDF statement.
Post-conditions:	Supplier ledger shows net debt Rs. 120,000; bank balance reduced by Rs. 80,000.

9. REQUIREMENTS TRACEABILITY MATRIX & CODE MAP

Requirement ID	Requirement Description	Primary Source Code File	Target Symbol / Method
REQ-01-01	Kiosk Mode Window Initialization	`lib/main.dart`	`main`, `windowManager`
REQ-02-01	Drift Schema Definition v11	`lib/database/database.dart`	`AppDatabase`, `migration`
REQ-02-02	Database Self-Healing Engine	`lib/cubits/erp_cubit.dart`	`loadData()`
REQ-02-03	Legacy Invoice Backfill	`lib/cubits/erp_cubit.dart`	`loadData()`
REQ-03-01	Fleet Status & Dashboard Metrics	`lib/screens/dashboard_screen.dart`	`DashboardScreen`
REQ-04-01	Traila Fleet Registration	`lib/cubits/erp_cubit.dart`	`addTraila()`
REQ-04-04	Traila Production Status Updates	`lib/cubits/erp_cubit.dart`	`updateTrailaStatus()`
REQ-05-01	Manufacturing Cost Aggregation	`lib/cubits/erp_cubit.dart`	`_syncTrailaTotalCost()`
REQ-06-02	Supplier Purchase Bill Entry	`lib/cubits/erp_cubit.dart`	`addSupplierPurchase()`
REQ-07-01	Supplier Dual-Balance Settlement	`lib/cubits/erp_cubit.dart`	`_syncSupplierBalances()`
REQ-07-03	Supplier Multi-Account Split Payment	`lib/cubits/erp_cubit.dart`	`_addSupplierPaymentInternal()`
REQ-08-02	Labour Work Order Logging	`lib/cubits/erp_cubit.dart`	`addLabourWork()`
REQ-09-01	Labour Wage Balance Reconciliation	`lib/cubits/erp_cubit.dart`	`_syncLabourBalances()`
REQ-10-01	Ready Unit Sale & Profit Engine	`lib/cubits/erp_cubit.dart`	`sellTraila()`
REQ-11-01	Buyer Consolidated Debt Tracking	`lib/cubits/erp_cubit.dart`	`sellTraila()`

Requirement ID	Requirement Description	Primary Source Code File	Target Symbol / Method
REQ-11-02	Sale Installment Settlement	`lib/cubits/erp_cubit.dart`	`addSalePayment()`
REQ-12-04	Khata Bidirectional Account Sync	`lib/cubits/erp_cubit.dart`	`addKhata()`
REQ-13-03	Atomic Double-Entry Fund Transfer	`lib/cubits/erp_cubit.dart`	`transferMoney()`
REQ-14-02	P&L Financial Engine Evaluation	`lib/screens/expense_revenue_screen.dart`	`_ExpenseRevenueScreenState`
REQ-15-01	Vector PDF Document Generation	`lib/services/invoice_pdf_service.dart`	`InvoicePdfService`
REQ-16-03	Thermal Monochrome Thresholding	`lib/services/unified_print_service.dart`	`_getThermalLogo()`
REQ-17-01	WhatsApp Automation & Explorer	`lib/services/whatsapp_service.dart`	`_sendWhatsAppWindows()`
REQ-18-01	Enterprise Branding Metadata	`lib/cubits/erp_cubit.dart`	`updateAppProfile()`

10. GLOSSARY & DOMAIN TERMINOLOGY

Term	Domain Definition & Operational Usage
Traila (ٹرالہ)	Heavy agricultural or commercial vehicular trailer, articulated trolley, or flatbed manufactured and sold by the workshop.
Khata (کھاتہ)	Traditional double-entry debit/credit ledger maintained for suppliers, labourers, or general business counter-parties.
Udhaar (ادھار)	Commercial credit extended to customers or credit balances owed to raw material suppliers.
Manufacturing Cost	Internal unit production expenditure calculated by aggregating materials, piece-rate labour charges, and direct expenses.
Advance Balance	Credit balance held in favor of the workshop when payments exceed billed procurement goods.
Kiosk Mode	Desktop window execution configuration where OS window decorations are stripped and the app runs auto-maximized.
ESC/POS	Standardized command protocol utilized by thermal receipt printers.
Drift ORM	Type-safe reactive persistence library for Dart/Flutter abstracting SQLite with schema migration management.
BLoC / Cubit	Business Logic Component pattern managing unidirectional reactive state flow.